

OIDC-Flows

Handout für Rollenspiel

Annett Laube
09.10.2025

HSLU Hochschule Luzern

Inhaltsverzeichnis

1	Einleitung	3
2	Akteure und Token	3
3	Authorization Code Flow	4
4	Session-Refresh	7
5	Logout	8
6	Abbildungsverzeichnis	9
7	Literaturverzeichnis	9

1 Einleitung

Das Dokument gibt einen Einblick in die grundlegenden Abläufe bei einer Authentifizierung (Login) mit OIDC, beim Session-Refresh bzw. beim Logout.

Die vorgestellten Abläufe (Flows) bieten die Grundlage, um die OIDC-Abläufe in einem Rollenspiel nachzustellen.

2 Akteure und Token

In diesem Dokument werden die ausgewählte OIDC-Flows näher beschrieben. Zuvor werden die verwendeten Akteure und Token kurz eingeführt.

2.1 Akteure

- **Benutzer (End User)**: Person, die auf eine Applikation zugreifen möchte, die eine OIDC-Authentifizierung fordert.
- **User Agent (Webbrowser)**
- **Client/Client Application**: Relying Party (RP), d.h. ein OAuth 2.0 Client, der einen Benutzer beim Server (OP) authentifizieren möchte und Informationen zu diesem Benutzer erlangen will.
- **(Authorization) Server**: IdP, der ein OIDC-Provider (OP) ist und verschiedene APIs (Endpoints) anbietet.

2.2 Token

Bei OIDC werden die folgenden Codes und Token verwendet:

- **Authorization Code**: ein vom Authorization Server generierter String (Hash-Wert):
  - ist meist ca. 10 min gültig,
  - sollte nur einmal verwendet werden,
  - ist an den Client-Identifikator und die Redirect-URI gebunden.

- **Access Token**: eine Zeichenkette, die einen bestimmten Bereich, eine bestimmte Lebensdauer und andere Zugriffsattribute codiert enthält. Ein Access Token ist ein Zugangstoken, dass dem jeweiligen Client erlaubt auf einen Server End Point (API) zuzugreifen.

- **Refresh Token:** enthalten eine Berechtigung, um neue Access Token für einen bestimmten Benutzer einzutauschen.

- **ID Token**: ist ein Sicherheits-Token (JWS), das Angaben über die Authentifizierung eines Endbenutzers durch einen Autorisierungsserver bei der Verwendung eines Clients und möglicherweise andere angeforderte Angaben enthält.

```
{
   "iss": "https://server.example.com",
   "sub": "24400320",
   "aud": "s6BhdRkqt3",
   "nonce": "n-0S6_WzA2Mj",
   "exp": 1311281970,
   "iat": 1311280970,
   "auth_time": 1311280969,
   "acr": "urn:mace:incommon:iap:silver"
  }
```

3 Authorization Code Flow

In Abbildung 1 ist der Ablauf des **OIDC Authorization Code Flow** schematisch dargestellt, im Folgenden werden die einzelnen Schritte mit passender Nummerierung und Beispielen beschrieben.

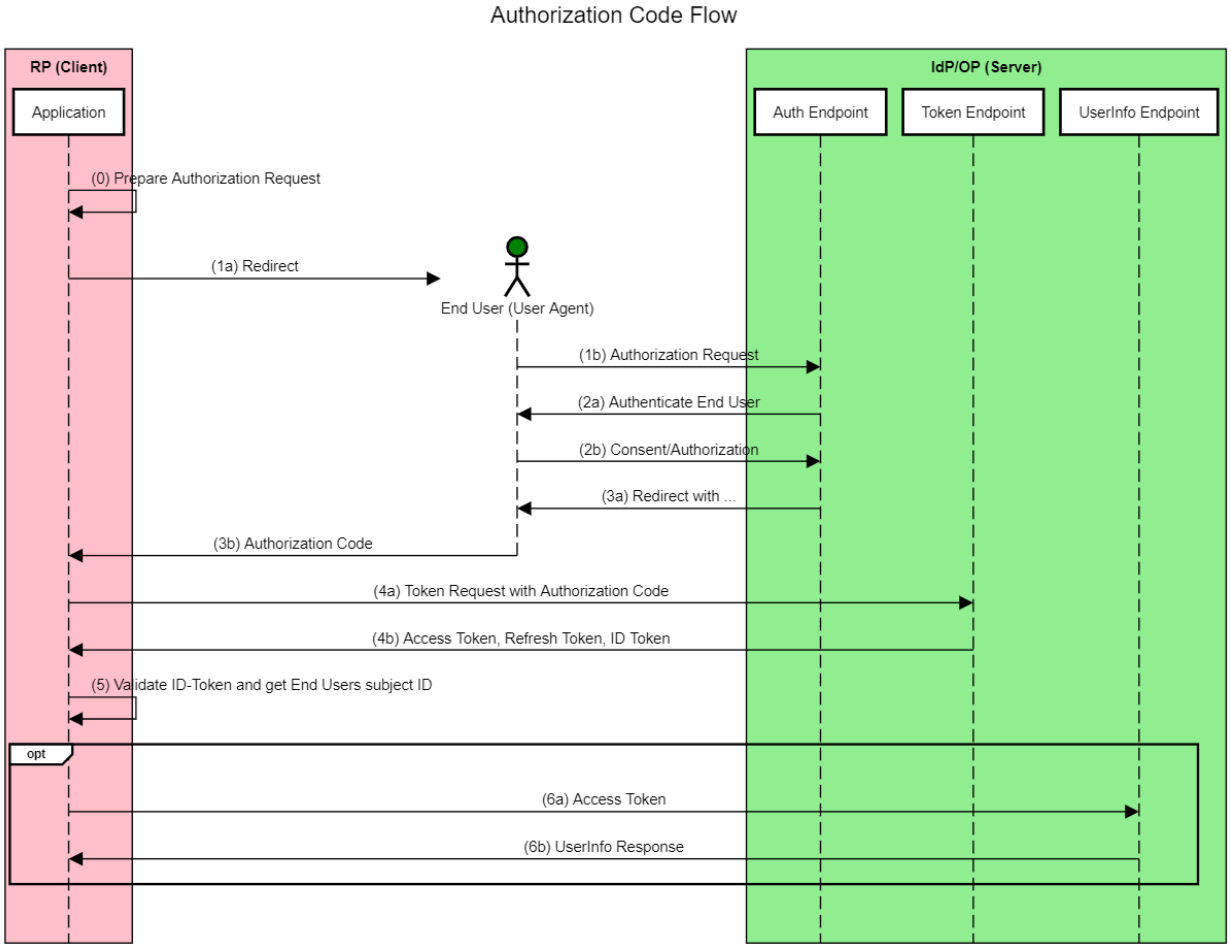


Abbildung 1: Ablauf beim Authorization Code Flow

0. Der Benutzer möchte auf eine Ressource bei `client.example.org` zugreifen.

1a. Wenn noch keine Benutzer-Session am Client (`client.example.org`) besteht, antwortet der Client mit dem folgenden Redirect-Anforderungen, die den User-Agent dazu bringt, einen Authentication Request an den Server abzusetzen.

```
HTTP/1.1 302 Found
Location: https://server.example.com/authorize?
response_type=code
&scope=openid%20profile%20email
&client_id=s6BhdRkqt3
&state=af0ifjsldkj
&redirect_uri=https%3A%2F%2Fclient.example.org%2Fcb
```

1b. Der User Agent schickt als Antwort auf den Redirect die folgende Authentifizierungsanfrage (Authentication Request) an den Authentication Endpoint des Server (`server.example.com`)

```
GET /authorize?
response_type=code
&scope=openid%20profile%20email
&client_id=s6BhdRkqt3
&state=af0ifjsldkj
&redirect_uri=https%3A%2F%2Fclient.example.org%2Fcb
HTTP/1.1
Host: server.example.com
```

1c. Der Autorisierungsserver MUSS die Anfrage validieren.

2a. Der Autorisierungsserver authentifiziert den Benutzer.

2b. Der Autorisierungsserver bekommt die Zustimmung (Autorisierung) des Benutzers.

3a. Der Autorisierungsserver sendet den Benutzer mit einem Autorisierungscode über den User Agent an den Client zurück.

Success:

```
HTTP/1.1 302 Found
Location: https://client.example.org/cb?
code=SplxlOBeZQQYbYS6WxSbIA
&state=af0ifjsldkj
```

Error:

```
HTTP/1.1 302 Found
Location: https://client.example.org/cb?
error=invalid_request
&error_description=
Unsupported%20response_type%20value
&state=af0ifjsldkj
```

3b. Der Client MUSS die Antwort validieren.

4a. Der Client fordert eine Antwort unter Verwendung des Autorisierungscodes am Token-Endpunkt an (per Backchannel). Für die Kommunikation mit dem Token-Endpunkt MUSS TLS verwendet werden. Der Client muss sich gegenüber dem Server authentifizieren (basierend auf einer vorgängigen Registrierung).

```
POST /token HTTP/1.1
Host: server.example.com
Content-Type: application/x-www-form-urlencoded
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW

grant_type=authorization_code&code=SplxlOBeZQQYbYS6WxSbIA
&redirect_uri=https%3A%2F%2Fclient.example.org%2Fcb
```

4b. Der Autorisierungsserver muss die Token-Anforderung validieren und schickt im Erfolgsfall ein ID Token, ein Access Token und optional ein Refresh Token an den Client zurück.

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
{
"access_token": "SlAV32hkKG",
"token_type": "Bearer",
"refresh_token": "8xLOxBtZp8",
"expires_in": 3600,
"id_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6IjFlOWdkazcifQ.ewogImlzc
yI6ICJodHRwOi8vc2VydmVyLmV4YW1wbGUuY29tIiwKICJzdWIiOiAiMjQ4Mjg5
NzYxMDAxIiwKICJhdWQiOiAiczZCaGRSa3F0MyIsCiAibm9uY2UiOiAibi0wUzZ
fV3pBMk1qIiwKICJleHAiOiAxMzExMjgxOTcwLAogImlhdCI6IDEzMTEyODA5Nz
AKfQ.ggW8hZ1EuVLuxNuuIJKX_V8a_OMXzR0EHR9R6jgdqrOOF4daGU96Sr_P6q
Jp6IcmD3HP99Obi1PRs-cwh3LO-p146waJ8IhehcwL7F09JdijmBqkvPeB2T9CJ
NqeGpe-gccMg4vfKjkM8FcGvnzZUN4_KSP0aAp1tOJ1zZwgjxqGByKHiOtX7Tpd
QyHE5lcMiKPXfEIQILVq0pc_E2DzL7emopWoaoZTF_m0_N0YzFC6g6EJbOEoRoS
K5hoDalrcvRYLSrQAZZKflyuVCyixEoV9GfNQC3_osjzw2PAithfubEEBLuVVk4
XUVrWOLrLl0nx7RkKU8NXNHq-rvKMzqg"
}
```

Oder eine Token Error Response:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
Cache-Control: no-store
{
"error": "invalid_request"
}
```

5. Der Client validiert das ID Token und das Access Token und ruft den Subject Identifier des Benutzers ab.

6a. (optional) Mit dem erhaltenen Access-Token kann der Client weitere Informationen zum Benutzer beim UserInfo Endpoint anfordern. Für die Kommunikation mit dem UserInfo Endpunkt MUSS TLS verwendet werden.

```
GET /userinfo HTTP/1.1
Host: server.example.com
Authorization: Bearer SlAV32hkKG
```

6b. Der Server validiert die Anfrage und sendet im Erfolgsfall die gewünschten Informationen (Claims) zum Benutzer zurück.

```
HTTP/1.1 200 OK
Content-Type: application/json
{"sub": "248289761001",
"name": "Jane Doe",
"given_name": "Jane",
"family_name": "Doe",
"preferred_username": "j.doe",
"email": "janedoe@example.com",
"picture": "http://example.com/janedoe/me.jpg"
}
```

4 Session-Refresh

In Abbildung 2 ist der Ablauf bei einem Session-Refresh schematisch dargestellt, im Folgenden werden die einzelnen Schritte mit passender Nummerierung und Beispielen beschrieben.

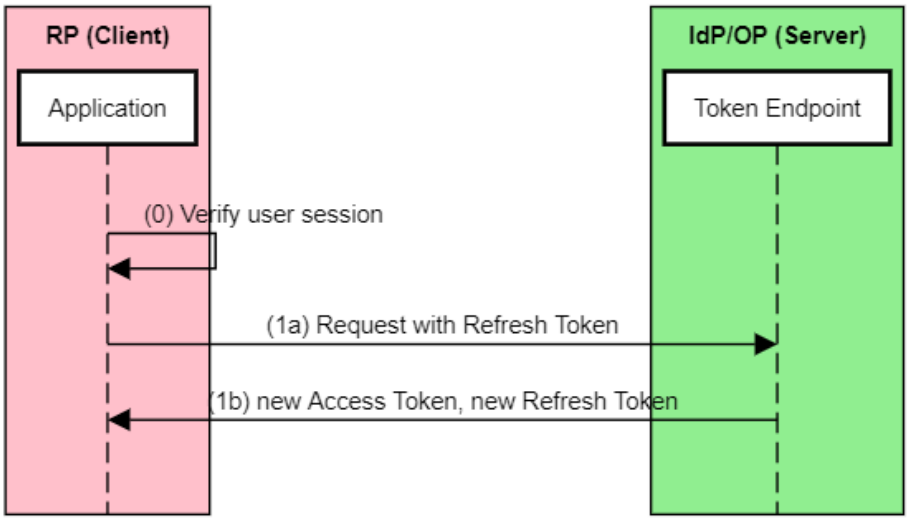


Abbildung 2: Ablauf beim Session-Refresh

0. Der Client möchte die Benutzer-Session verlängert. Er hat noch ein gültiges Refresh Token.

1a. Der Client schickt einen Token Request via HTTP POST (per BackChannel) zum Token-Endpunkt. Er schickt das noch gültige Refresh Token mit. Für die Kommunikation mit dem Token-Endpunkt MUSS TLS verwendet werden. Der Client muss sich gegenüber dem Server authentifizieren.

```
POST /token HTTP/1.1
Host: server.example.com
Content-Type: application/x-www-form-urlencoded
client_id=s6BhdRkqt3
  &client_secret=some_secret12345
  &grant_type=refresh_token
  &refresh_token=8xLOxBtZp8
  &scope=openid%20profile
```

1b. Der Server muss den Token Request validieren. Er überprüft die Gültigkeit des Refresh Tokens, ob es an den Client ausgestellt wurde und ob der Client sich erfolgreich authentifiziert hat. Nach erfolgreicher Validierung gibt der Server ein neues Access Token und eine neues Refresh Token an den Client zurück.

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
{
 "access_token": "TlBN45jURg",
 "token_type": "Bearer",
 "refresh_token": "9yNOxJtZa5",
 "expires_in": 3600
}
```

5 Logout

In Abbildung 3 ist der Ablauf des **Back-Channel Logouts** schematisch dargestellt. In der Abbildung wird der Server als IdP und der Client als RP bezeichnet.
Im Folgenden werden die einzelnen Schritte mit passender Nummerierung und Beispielen beschrieben.

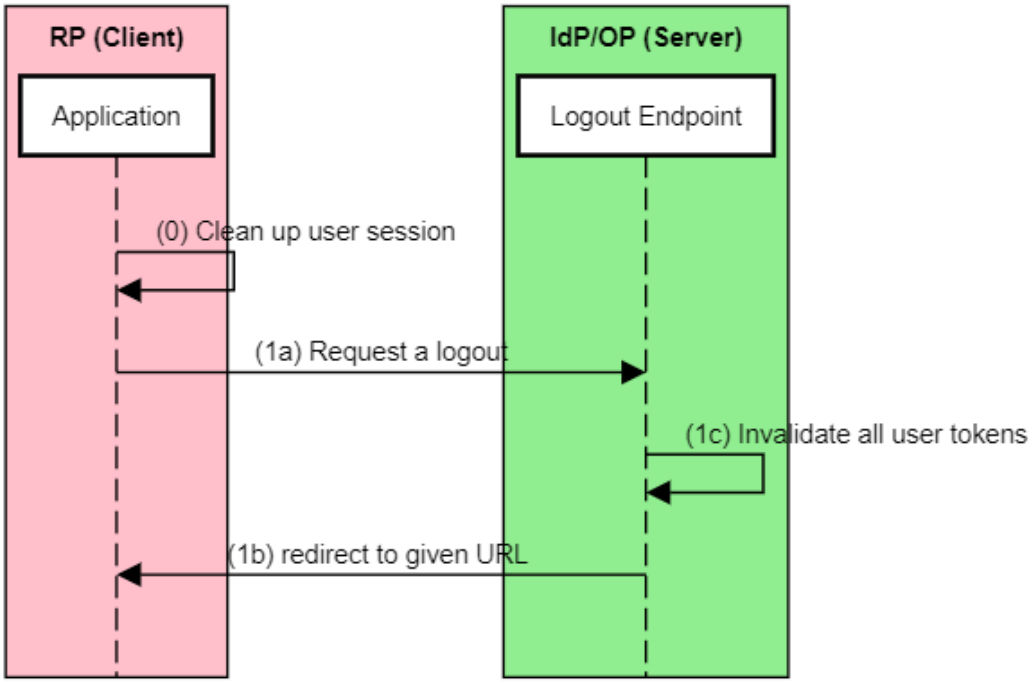


Abbildung 3: OIDC-Logout initiiert durch die RP

Ausgangssituation: der Benutzer hat bei einer Applikation (Client) mit Hilfe eines IdP/OP (Server) eingeloggt und möchte sich nun bei dieser (einen) Applikation ausloggen.
1a. Der Benutzer initiiert den Logout beim Client.
1b. Der Client invalidiert die Benutzersession und räumt auf. Es werden alle Session bezogenen Informationen im Browser (Cookies, HTML5 local stoarge,...) gelöscht. Der Client initiiert einen Redirect zum IdP, um dort ebenfalls die Benutzersession zu beenden. `GET https://server.example.com/oidc/logout?id_token_hint=previvously_issued_jwt_id_token& post_logout_redirect_uri=https://app1.levvel.io/oidc/logged_out& state=some_random_string` Es ist empfohlen, das ID Token des Benutzers mitzuschicken. Ausserdem muss eine URL mitgeben werden, an die der Benutzer nach dem Logout geschickt wird.
1c. Der Server terminiert die Benutzersession und revoziert alle entsprechenden Tokens. Er ruft über mittels Redirect über den Webbrowser die mitgeschickte URL auf, die dann dem Benutzer das erfolgreiche Logout signalisiert.

Abbildungsverzeichnis

Abbildung 1: Ablauf beim Authorization Code Flow	4
Abbildung 2: Ablauf beim Session-Refresh	7
Abbildung 3: OIDC-Logout initiiert durch die RP	8

6 Literaturverzeichnis

OpenID Connect Core 1.0
https://openid.net/specs/openid-connect-core-1_0.html

RFC 6749 - The OAuth 2.0 Authorization Framework
https://www.rfc-editor.org/info/rfc6749